2026 ENTERPRISE EDITION

SQL & DATABASES

DATTA SABLE TECHNICAL LIBRARY

PostgreSQL High-Performance Tuning & DBA Guide

The comprehensive production manual for database administrators and senior engineers: shared_buffers sizing, autovacuum optimization, EXPLAIN BUFFERS mastery, PgBouncer pooling, and disaster recovery.

22
Production Chapters & Architecture Blueprints

100%
Practical Code, Schemas & Frameworks

2026
Modern Enterprise Best Practices

INDEPENDENTLY AUTHORED BY

Datta Sable
Enterprise BI & Data Architect

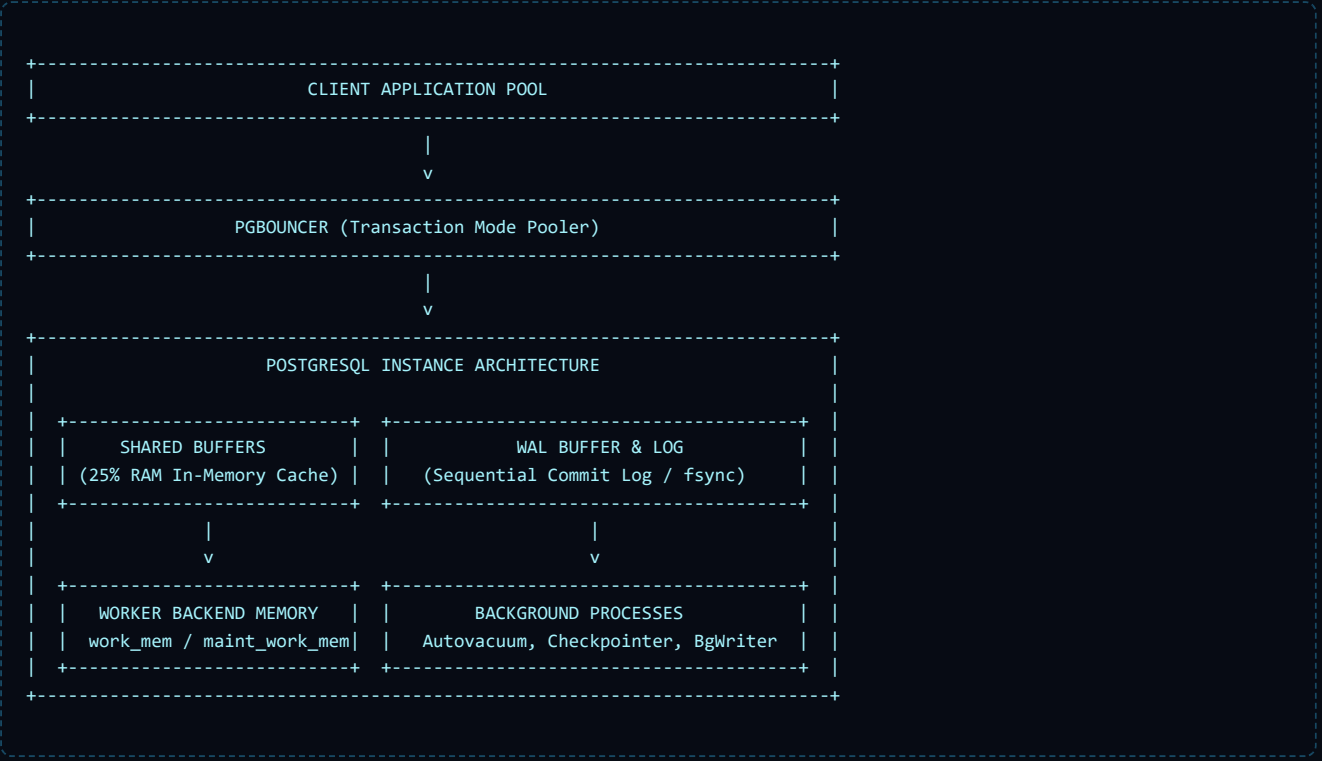
Published: October 2026
dattasable.com/ebooks

ARCHITECTURE BLUEPRINT

Table of Contents & PostgreSQL Node Topology

EXECUTIVE OVERVIEW

PostgreSQL powers mission-critical enterprise workloads globally. Operating PostgreSQL at high throughput (50,000+ transactions per second) requires deep mastery of shared memory structures, disk write-ahead log mechanics, indexing strategies, and automated connection pooling.



Comprehensive Chapter Guide

Part I: Engine & Storage

- Ch 3: Memory & shared_buffers
- Ch 4: WAL & Checkpoint Sizing
- Ch 5: Autovacuum & Bloat Mitigation
- Ch 6: Index Types (B-Tree, BRIN, GIN)
- Ch 7: Covering & Partial Indexes

Part II: Execution & Scale

- Ch 8: EXPLAIN BUFFERS Mastery
- Ch 9: Join Spills & Hash Work Mem
- Ch 10: Declarative Partitioning
- Ch 11: PgBouncer Pooling Topology
- Ch 12: Locks & Deadlock Defense

Part III: Enterprise Reliability: Ch 13 Streaming & Logical Replication, Ch 14 Patroni HA Failover, Ch 15 pgBackRest & PITR Recovery, Ch 16 pg_stat_statements & Plan Regression, Ch 17 Security Hardening & RLS, Ch 18 Linux Kernel sysctl Sizing, Ch 19 Corruption Recovery Case Study, Ch 20 Anti-Patterns, Ch 21 Emergency Runbook, Ch 22 Production Health Checklist.

CHAPTER 01

Memory Architecture: shared_buffers & work_mem

MEMORY SIZING INTERNALS

PostgreSQL relies on a dual-caching architecture: its internal shared_buffers cache works in tandem with the Linux operating system kernel page cache. Allocating memory requires balancing shared pools with per-query worker allocations.

The Memory Allocation Triad

- **shared_buffers:** Set to 25% of total server RAM for dedicated Linux database hosts (e.g. 16GB on a 64GB node). Setting it above 40% degrades performance due to double-buffering overhead and Linux page eviction contention.
- **work_mem:** Memory allocated to *each individual sort, hash join, and aggregation operator* within a query. A query with 4 joins can consume 4x work_mem. Formula: `((Total RAM - shared_buffers) * 0.8) / (max_connections * 2)`.
- **maintenance_work_mem:** Memory utilized during VACUUM, CREATE INDEX, and ALTER TABLE operations. Set to 1GB – 2GB on production systems to accelerate index builds.

postgresql.conf

```
# Production Memory Configuration for 64GB RAM Node
shared_buffers = 16GB
effective_cache_size = 48GB
work_mem = 64MB
maintenance_work_mem = 2GB
max_connections = 200
temp_buffers = 32MB
```

OOM KILLER ALERT

If `max_connections = 1000` and `work_mem = 128MB`, a burst of complex queries can demand 128GB of RAM, triggering the Linux Out-Of-Memory (OOM) killer to terminate the PostgreSQL postmaster process instantly.

CHAPTER 02

Write-Ahead Logging & Checkpoint Tuning

TRANSACTION DURABILITY

PostgreSQL writes all database modifications sequentially to the Write-Ahead Log (WAL) before updating data pages in shared_buffers. A Checkpoint flushes dirty memory buffers to disk to bound crash recovery time.

Infrequent checkpoints maximize write throughput by smoothing I/O spikes, while overly frequent checkpoints trigger I/O saturation and write amplification.

Configuration Parameter	Default Setting	Production Value	Architectural Justification
max_wal_size	1GB	32GB – 64GB	Prevents premature checkpoint triggers during bulk ingestion
min_wal_size	80MB	4GB – 8GB	Re-uses pre-allocated WAL segments to avoid filesystem churn
checkpoint_completion_target	0.9	0.9	Spreads buffer writes across 90% of checkpoint interval
checkpoint_timeout	5min	15min – 30min	Reduces checkpoint frequency while maintaining recovery bounds

Diagnosing Checkpoint Frequency

checkpoint_diagnostics.sql

```sql
-- Check if checkpoints are triggered by timeout or wal volume
SELECT
    checkpoints_timed,
    checkpoints_req,
    checkpoint_write_time,
    checkpoint_sync_time,
    ROUND(checkpoints_req * 100.0 / (checkpoints_timed + checkpoints_req), 2) AS pct_requested_checkpoints
FROM pg_stat_bgwriter;
```

TUNING RULE

If `pct_requested_checkpoints` exceeds 10%, PostgreSQL is constantly running out of WAL capacity before reaching `checkpoint_timeout`. Increase `max_wal_size` immediately.

CHAPTER 03

Vacuuming, Bloat Mitigation & Autovacuum

MVCC MAINTENANCE

PostgreSQL's Multi-Version Concurrency Control (MVCC) never overwrites existing rows in-place. An UPDATE marks the original row as 'dead' and writes a new row version. Autovacuum reclaims dead tuple space and prevents transaction ID wraparound.

Default autovacuum settings are famously conservative. On tables with 50 million rows, waiting for 20% of rows to die before triggering vacuum leads to catastrophic table bloat and disk exhaustion.

autovacuum_production.conf

```
# High-Throughput Autovacuum Tuning in postgresql.conf
autovacuum = on
autovacuum_max_workers = 6
autovacuum_naptime = 15s
autovacuum_vacuum_threshold = 50
autovacuum_vacuum_scale_factor = 0.05       # Trigger after 5% change instead of 20%
autovacuum_vacuum_cost_limit = 2000         # Increase I/O budget (default is 200)
autovacuum_vacuum_cost_delay = 2ms          # Reduce throttling sleep delay
```

Table-Specific Autovacuum Overrides

For ultra-high-velocity tables (e.g. `sessions`, `queue_jobs`), configure table-level storage parameters:

table_vacuum_override.sql

```
ALTER TABLE public.active_user_sessions SET (
    autovacuum_vacuum_scale_factor = 0.01,
    autovacuum_vacuum_threshold = 100,
    autovacuum_vacuum_cost_limit = 5000
);
```

TRANSACTION ID WRAPAROUND

Never disable autovacuum. If the system approaches 2 billion transactions without vacuum freezing old tuples, PostgreSQL enters read-only emergency recovery mode to prevent data corruption.

CHAPTER 04

Indexing Deep Dive: B-Tree, GIN, GiST & BRIN

ACCESS METHOD SELECTION

Selecting the appropriate index access method transforms multi-second table scans into sub-millisecond lookups while avoiding index maintenance overhead.

Index Access Method	Underlying Data Structure	Optimal Workload Fit	Storage Size Footprint
B-Tree (Default)	Self-balancing search tree	Equality (=) and range (<, >, BETWEEN) on unique/high cardinality keys	Medium (Increases with table size)
GIN (Generalized Inverted)	Inverted index mapping elements to rows	Full-text search, arrays, JSONB ?, @> containment operators	Large (Can exceed base table size)
GiST (Generalized Search)	Lossy tree index	Geometric shapes, PostGIS geospatial coordinates, range intervals	Medium
BRIN (Block Range Index)	Min/max value summary per disk page block	Append-only naturally sorted time-series or sequential IDs	Tiny (< 1% of B-Tree size)

Creating BRIN Indexes on Multi-Terabyte Tables

brin_index_creation.sql

```sql
-- 100M row audit table: B-Tree index = 2.4GB; BRIN index = 4MB!
CREATE INDEX idx_audit_log_created_at_brin
ON logging.audit_events
USING BRIN (created_at) WITH (pages_per_range = 128);
```

CHAPTER 05

Covering Indexes, Partial Indexes & Expressions

ADVANCED INDEXING

Advanced indexing techniques allow the PostgreSQL planner to satisfy queries directly from the index structure (Index-Only Scans) without touching underlying table heap blocks.

Covering Indexes with the INCLUDE Clause

A covering index appends non-key payload columns to the B-Tree leaf pages without incorporating them into the search tree traversal logic. This enables Index-Only Scans while preserving low index maintenance overhead.

covering_and_partial_indexes.sql

```sql
-- Covering Index: Index-Only Scan for user lookups by email
CREATE INDEX idx_users_email_covering
ON public.users (email)
INCLUDE (first_name, last_name, status, role);

-- Partial Index: Only index active uncompleted orders (1% of table)
CREATE INDEX idx_orders_unprocessed
ON public.orders (created_at)
WHERE status IN ('PENDING', 'PROCESSING');

-- Expression-based Index: Accelerated case-insensitive search
CREATE INDEX idx_users_lower_username
ON public.users (LOWER(username));
```

Why Partial Indexes Slash I/O

If an e-commerce database stores 10 million historical fulfilled orders and 10,000 active processing orders, an index on `status` indexes all 10 million rows. A partial index with `WHERE status = 'PENDING'` indexes only 10,000 rows, consuming 500KB of RAM instead of 250MB.

CONCURRENT BUILDS

Always specify `CREATE INDEX CONCURRENTLY` in production environments. Standard index creation locks the table against all write operations (INSERT/UPDATE/DELETE) until completion.

CHAPTER 06

EXPLAIN (ANALYZE, BUFFERS) Mastery

QUERY PLAN ANALYSIS

`EXPLAIN (ANALYZE, BUFFERS)` is the DBA's primary diagnostic weapon. While `ANALYZE` measures actual execution wall-clock time, `BUFFERS` reveals physical disk reads and shared memory cache hits.

query_plan_diagnostic.sql

```sql
EXPLAIN (ANALYZE, BUFFERS, VERBOSE, SETTINGS)
SELECT c.customer_name, SUM(o.total_amount)
FROM customers c
INNER JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_date >= '2026-01-01'
GROUP BY c.customer_name;
```

Key Plan Metrics Decoded

- **Buffers: shared hit=42100 read=120:** `hit` indicates pages found in PostgreSQL RAM; `read` indicates pages fetched from physical OS disk. High reads indicate cache eviction or cold data.
- **Actual Rows vs Plan Rows:** If the planner estimates `rows=10` but actual execution processes `rows=500000`, optimizer statistics are stale. Re-run `ANALYZE` immediately.
- **Seq Scan vs Index Scan:** Sequential scans are fast on small tables (< 1,000 rows), but devastating when scanning 10GB heap tables to return 3 records.

COST FORMAT

In `cost=0.42..8.45`, `0.42` is the startup cost before the first row is output; `8.45` is the total estimated cost to complete the node.

CHAPTER 07

Join Strategies: Nested Loop, Hash & Merge

ALGORITHMIC JOIN SELECTION

The query planner evaluates three fundamental join algorithms based on estimated row cardinality, available indexes, and memory constraints.

Join Strategy	Algorithmic Mechanism	Ideal Scenario	Risk Factor
Nested Loop	For each outer row, search inner table via index	Small outer set (< 1,000 rows) joining indexed inner table	Exponential O(N*M) slowdown if inner table lacks index
Hash Join	Build in-memory hash table of inner set; probe with outer	Large, unsorted tables with equality join conditions	Spills to disk (Batches > 1) if hash table exceeds work_mem
Merge Join	Sort both inputs by join key; advance pointers in lockstep	Large pre-sorted datasets (e.g. via B-Tree index)	Expensive sort phase if inputs are not already ordered

Detecting Hash Join Disk Spills

When an in-memory hash table exceeds `work_mem`, PostgreSQL splits the hash table into multiple batches and spills chunks to temporary disk files. In EXPLAIN output, look for:

explain_spill_output.txt

```
Hash (cost=420.00..420.00 rows=25000 width=16)
  Buckets: 32768  Batches: 4  Memory Usage: 1024kB  Disk Usage: 4280kB
```

Remediation: Increase `work_mem` for the session or optimize the join predicate to prevent temporary disk write amplification.

CHAPTER 08

Declarative Partitioning: Range, List & Hash

PHYSICAL HORIZONTAL SCALING

Declarative partitioning splits massive tables (e.g. > 100GB) into smaller physical tables managed under a single parent logical entity, dramatically accelerating query pruning and index maintenance.

Partitioning Typologies

- **RANGE:** Partitioned by continuous numeric or timestamp boundaries (e.g. Monthly orders).
- **LIST:** Partitioned by explicit categorical keys (e.g. Country code: US, EMEA, APAC).
- **HASH:** Uniformly distributes rows across N modulus shards for even distribution.

declarative_range_partitioning.sql

```sql
-- Parent Partitioned Table
CREATE TABLE billing.invoices (
    invoice_id BIGINT,
    customer_id INT,
    invoice_date DATE NOT NULL,
    total_amount DECIMAL(12,2)
) PARTITION BY RANGE (invoice_date);

-- Child Partitions for 2026 Quarters
CREATE TABLE billing.invoices_2026_q1 PARTITION OF billing.invoices
    FOR VALUES FROM ('2026-01-01') TO ('2026-04-01');

CREATE TABLE billing.invoices_2026_q2 PARTITION OF billing.invoices
    FOR VALUES FROM ('2026-04-01') TO ('2026-07-01');
```

Run-Time Partition Pruning

Ensure `enable_partition_pruning = on` in `postgresql.conf`. When queries include `WHERE invoice_date = '2026-02-15'`, the planner prunes all partitions except `invoices_2026_q1`, avoiding table scans across 95% of database storage.

INSTANT DATA PURGE

Dropping an old monthly partition via `DROP TABLE billing.invoices_2024_q1` executes in 5 milliseconds with zero table bloat, completely bypassing slow `DELETE WHERE date < ...` commands.

CHAPTER 09

Connection Pooling: PgBouncer Architecture

CONCURRENCY SCALING

PostgreSQL allocates a dedicated operating system process (fork) for every connected client. Maintaining 2,000 direct database connections exhausts server RAM and causes CPU context-switching starvation.

PgBouncer sits between applications and PostgreSQL, multiplexing thousands of application connections into a lean pool of 50 – 100 persistent database backend connections.

PgBouncer Pool Mode	Connection Lifecycle	Prepared Statements	Production Suitability
Session Mode	Server connection held until client explicitly disconnects	Fully supported	Low-scale; behaves like direct connections
Transaction Mode	Server connection released back to pool when transaction ends	Supported via named protocol in PgBouncer 1.21+	**Optimal** (95% of microservices)
Statement Mode	Server connection released immediately after single statement	Not supported	Autocommit only (Multi-statement txns fail)

Production pgbouncer.ini Configuration

pgbouncer.ini

```
[databases]
production_db = host=127.0.0.1 port=5432 dbname=production_db pool_size=50

[pgbouncer]
listen_port = 6432
listen_addr = *
auth_type = scram-sha-256
auth_file = /etc/pgbouncer/userlist.txt
pool_mode = transaction
max_client_conn = 5000
default_pool_size = 40
reserve_pool_size = 10
reserve_pool_timeout = 5.0
```

CHAPTER 10

Concurrency & Lock Management: Deadlocks

CONCURRENCY DEFENSE

High-concurrency systems encounter transaction lock contention. Diagnosing deadlocks, row-level locks, and table locks prevents multi-minute application outages.

Lock Hierarchy & Conflicts

PostgreSQL defines 8 table lock modes. Two transactions requesting mutually conflicting lock levels on the same resource cause one transaction to block. If Transaction A waits for Transaction B while B waits for A, a **Deadlock** occurs.

lock_investigation.sql

```sql
-- Real-time blocked vs blocking query detection
SELECT
    blocked_locks.pid     AS blocked_pid,
    blocked_activity.usename AS blocked_user,
    blocking_locks.pid    AS blocking_pid,
    blocking_activity.usename AS blocking_user,
    blocked_activity.query    AS blocked_statement,
    blocking_activity.query   AS current_statement_in_blocking_process
FROM  pg_catalog.pg_locks         blocked_locks
JOIN pg_catalog.pg_stat_activity blocked_activity ON blocked_activity.pid = blocked_locks.pid
JOIN pg_catalog.pg_locks         blocking_locks
    ON blocking_locks.locktype = blocked_locks.locktype
    AND blocking_locks.database IS NOT DISTINCT FROM blocked_locks.database
    AND blocking_locks.relation IS NOT DISTINCT FROM blocked_locks.relation
    AND blocking_locks.page IS NOT DISTINCT FROM blocked_locks.page
    AND blocking_locks.tuple IS NOT DISTINCT FROM blocked_locks.tuple
    AND blocking_locks.virtualxid IS NOT DISTINCT FROM blocked_locks.virtualxid
    AND blocking_locks.transactionid IS NOT DISTINCT FROM blocked_locks.transactionid
    AND blocking_locks.classid IS NOT DISTINCT FROM blocked_locks.classid
    AND blocking_locks.objid IS NOT DISTINCT FROM blocked_locks.objid
    AND blocking_locks.objsubid IS NOT DISTINCT FROM blocked_locks.objsubid
    AND blocking_locks.pid != blocked_locks.pid
JOIN pg_catalog.pg_stat_activity blocking_activity ON blocking_activity.pid = blocking_locks.pid
WHERE NOT blocked_locks.granted;
```

DEADLOCK MITIGATION RULE

Always acquire row locks in the exact same deterministic order across application code (e.g. `SELECT ... FOR UPDATE ORDER BY id ASC`) to eliminate cyclical deadlock conditions.

CHAPTER 11

Replication: Physical Streaming vs. Logical

DATA PROPAGATION

PostgreSQL offers two distinct replication architectures: Physical Streaming Replication (exact binary replica of entire database cluster) and Logical Replication (publish/subscribe on individual tables).

Replication Parameter	Physical Streaming Replication	Logical Replication (Pub/Sub)
Replication Unit	Entire cluster (All databases & tables)	Granular tables or schemas
Target Writeability	Read-Only Standby	Read/Write (Can insert local data)
Cross-Version Upgrades	Impossible (Exact major version match)	Supported (e.g. PG 15 -> PG 17 zero-downtime)
Local Indexes / Triggers	Identical to primary	Can create custom indexes on subscriber

Configuring Logical Publication & Subscription

logical_replication_setup.sql

```sql
-- On Primary Server (Publisher)
CREATE PUBLICATION pub_analytics_feed
FOR TABLE orders, order_items, customers;

-- On Analytics Replica Server (Subscriber)
CREATE SUBSCRIPTION sub_analytics_feed
CONNECTION 'host=primary.prod.internal port=5432 dbname=prod_db user=rep_user password=SecretPassword'
PUBLICATION pub_analytics_feed;
```

REPLICATION LAG WARNING

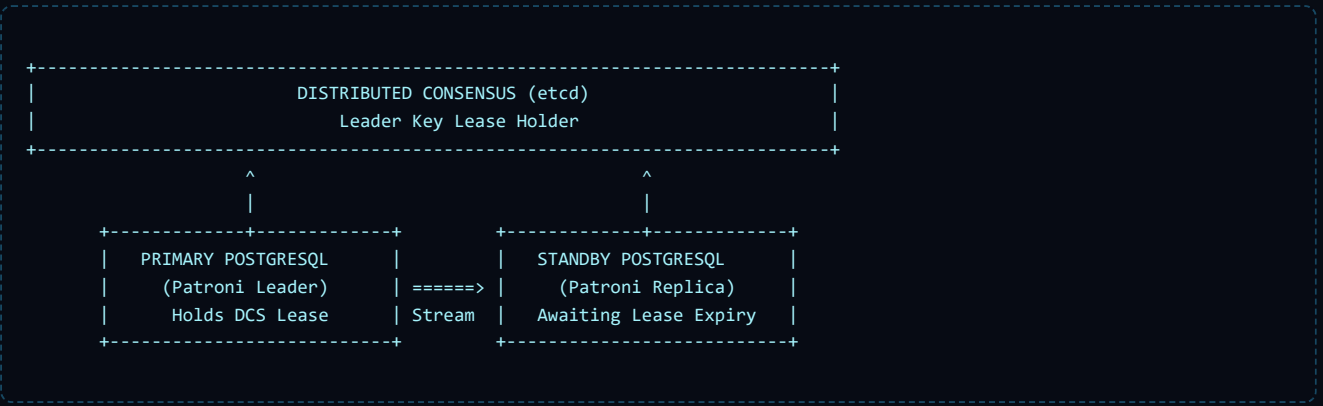
Monitor `pg_stat_replication` on the primary. If `write_lag` or `replay_lag` exceeds 30 seconds, read-replicas will serve stale data to reporting applications.

CHAPTER 12

High Availability & Automatic Failover: Patroni

ZERO-DOWNTIME TOPOLOGY

Enterprise SLAs demand automatic failover without split-brain scenarios. Patroni orchestrates high-availability PostgreSQL clusters using a Distributed Consensus Store (DCS) such as etcd or Consul.



How Patroni Prevents Split-Brain Disasters

In standard replication setups, a network partition can trick a standby into promoting itself while the old primary continues accepting writes. Patroni enforces leader key leasing in etcd with TTL heartbeats (10s). If the leader fails to renew its lease, it immediately demotes itself to read-only before the DCS elects a standby.

HA Failover Timeline

1. Primary node encounters hardware panic or kernel lockup.
2. etcd leader key lease expires after 10 seconds.
3. Standby with the lowest replication lag wins the consensus election.
4. Patroni promotes new primary, issues timeline switch, and updates health check endpoints for HAProxy/Keepalived routing. Total downtime: < 15 seconds.

CHAPTER 13

Backup & Point-in-Time Recovery: pgBackRest

DISASTER RECOVERY

pg_dump is a logical text export unsuitable for large multi-terabyte production databases. Enterprise disaster recovery requires physical binary backups with continuous WAL archiving via pgBackRest.

Why pgBackRest Outperforms pg_dump

- **Parallel Multi-Threaded Compression:** Compresses database files in parallel directly to AWS S3, Azure Blob, or local NAS storage.
- **Differential & Incremental Backups:** Copies only blocks changed since the last full or differential backup, reducing backup times from 6 hours to 10 minutes.
- **Point-in-Time Recovery (PITR):** Allows restoring the database to the exact millisecond before a disastrous `DROP TABLE` occurred.

pgbackrest.conf

```
[global]
repo1-type=s3
repo1-s3-bucket=enterprise-pg-backups
repo1-s3-endpoint=s3.us-east-1.amazonaws.com
repo1-s3-region=us-east-1
repo1-path=/backups/production
process-max=8
compress-type=lz4
compress-level=3

[production_stanza]
pg1-path=/var/lib/postgresql/16/main
pg1-user=postgres
```

Executing Point-in-Time Recovery

pitr_recovery.sh

```
# Restore cluster to 1 second prior to accidental data deletion
pgbackrest --stanza=production_stanza --delta \
  --type=time "--target=2026-10-28 14:29:59+00" \
  --target-action=promote restore
```

CHAPTER 14

Statistics & pg_stat_statements Profiling

OBSERVABILITY

pg_stat_statements records execution statistics for all SQL queries executed against the cluster. It is the single most valuable extension for identifying query plan regressions and performance bottlenecks.

Enabling pg_stat_statements

Add to postgresql.conf: shared_preload_libraries = 'pg_stat_statements' and run CREATE EXTENSION pg_stat_statements;.

top_cumulative_queries.sql

```sql
-- Find the top 5 queries consuming the most total cluster execution time
SELECT
    queryid,
    SUBSTRING(query, 1, 60) AS short_query,
    calls,
    ROUND(total_exec_time::numeric, 2) AS total_time_ms,
    ROUND(mean_exec_time::numeric, 2) AS mean_time_ms,
    ROUND((100.0 * total_exec_time / SUM(total_exec_time) OVER())::numeric, 2) AS pct_cluster_time,
    rows AS total_rows
FROM pg_stat_statements
ORDER BY total_exec_time DESC
LIMIT 5;
```

Planner Statistics & default_statistics_target

The query planner builds histograms and most common values (MCV) lists during ANALYZE. The default target (100) is often too coarse for skewed data. Increase it on critical columns:

column_statistics_boost.sql

```sql
ALTER TABLE orders ALTER COLUMN customer_id SET STATISTICS 1000;
ANALYZE orders(customer_id);
```

CHAPTER 15

Security Hardening: pg_hba.conf & Row Security

DEFENSE-IN-DEPTH

Hardening PostgreSQL against intrusion demands strict host-based authentication (pg_hba.conf), mandatory SSL/TLS encryption, SCRAM-SHA-256 password hashing, and granular Row-Level Security (RLS).

Production pg_hba.conf Lockdown

Never permit `trust` or plaintext `md5` authentication in production. Enforce SCRAM-SHA-256 over TLS:

pg_hba.conf

```
# TYPE  DATABASE        USER            ADDRESS                 METHOD
local   all             postgres                                peer
hostssl all             all             10.0.0.0/16             scram-sha-256
hostssl replication     rep_user        10.0.1.0/24             scram-sha-256
host    all             all             all                     reject
```

Implementing Row-Level Security (RLS)

RLS enforces tenant isolation at the database engine layer, guaranteeing that multi-tenant applications cannot accidentally leak cross-organization customer records even if application code is compromised.

tenant_rls_policy.sql

```sql
ALTER TABLE accounts.invoices ENABLE ROW LEVEL SECURITY;

CREATE POLICY tenant_isolation_policy ON accounts.invoices
    FOR ALL
    USING (tenant_id = current_setting('app.current_tenant_id')::INT);
```

LEAST PRIVILEGE

Revoke `PUBLIC` permissions on the `public` schema: `REVOKE ALL ON SCHEMA public FROM PUBLIC;` to prevent unprivileged users from creating rogue tables.

CHAPTER 16

Hardware Sizing & Linux OS Kernel sysctl Tuning

OS & HARDWARE OPTIMIZATION

PostgreSQL runs directly on the Linux operating system kernel. Suboptimal kernel parameters (overcommit behavior, swapiness, and huge pages) can cripple database performance despite powerful hardware.

Production sysctl.conf Configuration

/etc/sysctl.d/99-postgresql.conf

```
# Disable aggressive swapping (keep dirty pages in RAM longer)
vm.swappiness = 1

# Prevent Linux OOM killer from panicking during burst allocations
vm.overcommit_memory = 2
vm.overcommit_ratio = 80

# File descriptor and socket connection bounds
fs.file-max = 2097152
net.core.somaxconn = 4096
net.ipv4.tcp_max_syn_backlog = 4096
```

Huge Pages: Squeezing 15% Free Performance

On servers with > 32GB of RAM, default 4KB memory pages create enormous Translation Lookaside Buffer (TLB) page table overhead. Allocating **Transparent Huge Pages (disabled)** and configuring **Static Huge Pages (2MB)** eliminates TLB cache misses.

huge_pages_setup.sh

```
# In postgresql.conf
huge_pages = on

# In Linux sysctl
vm.nr_hugepages = 8200 # (shared_buffers 16GB / 2MB per page = 8192 pages)
```

CHAPTER 17

Disaster Recovery Case Study: Index Corruption

PRODUCTION EMERGENCY

A financial payment platform experienced an unexpected storage controller hardware failure, causing B-Tree index page corruption on the primary transactions table and halting payment processing.

Incident Symptoms & Diagnosis

Application logs threw: `ERROR: right sibling's left-link doesn't match: block 14201 in index "idx_trans_pk"`. Queries querying rows via the index failed while sequential scans succeeded.

Step-by-Step Production Resolution

1. **Isolate Traffic:** Instructed PgBouncer to pause the connection pool: `PAUSE transactions_db;`.
2. **Verify Physical Heap Integrity:** Executed `amcheck` extension to confirm heap data was uncorrupted:

amcheck_verification.sql

```sql
CREATE EXTENSION amcheck;
SELECT bt_index_check('idx_trans_pk', true);
```

3. **Rebuild Damaged Index Without Locking:**

reindex_recovery.sql

```sql
-- Rebuild corrupted primary key concurrently without blocking concurrent reads
REINDEX INDEX CONCURRENTLY idx_trans_pk;
```

4. **Resume Application Traffic:** Unpaused PgBouncer: `RESUME transactions_db;`. Total downtime: 4 minutes.

CHAPTER 18

PostgreSQL Anti-Patterns & Performance Traps

PITFALL PREVENTION

Common developer coding habits and ORM abstractions inadvertently bypass PostgreSQL optimizations, triggering severe CPU spikes and lock serialization.

Anti-Pattern	Underlying Failure Mechanism	Remediation Architecture
N+1 ORM Queries	ORM executes 1 query for parent, then 1,000 individual queries for each child row	Use `JOIN FETCH` or eager loading to retrieve child sets in a single query
Unindexed Foreign Keys	PostgreSQL does NOT auto-index foreign keys. Deleting a parent row triggers a full sequential scan on the child table!	Create B-Tree indexes on every foreign key column immediately
Offset Pagination	`OFFSET 1000000 LIMIT 20` reads 1,000,020 rows from disk and discards the first million!	Use Keyset Pagination: `WHERE id > :last_seen_id ORDER BY id ASC LIMIT 20`
SELECT * Everything	Forces heap fetches, disabling Index-Only Scans and saturating network I/O	Explicitly project only required columns in SELECT statements

The Sequence Exhaustion Bug

Creating tables with `INT` (32-bit integer) primary keys caps at 2,147,483,647 rows. When the sequence reaches this ceiling, all subsequent INSERTs fail. **Always use `BIGINT` for primary keys.**

CHAPTER 19

Emergency DBA Troubleshooting Runbook

CRITICAL INCIDENT RUNBOOK

When CPU reaches 100% or connection pools saturate, follow these emergency triage commands to restore service immediately.

emergency_triage_commands.sql

```sql
-- 1. Identify long-running rogue queries (> 60 seconds)
SELECT pid, now() - pg_stat_activity.query_start AS duration, query, state
FROM pg_stat_activity
WHERE (now() - pg_stat_activity.query_start) > interval '60 seconds'
AND state != 'idle';

-- 2. Gracefully cancel a runaway query (sends SIGINT)
SELECT pg_cancel_backend(14205);

-- 3. Forcefully terminate an un-cancelable zombie backend (sends SIGTERM)
SELECT pg_terminate_backend(14205);

-- 4. Check for disk space saturation on WAL directory
SELECT pg_size_pretty(sum(size)) FROM pg_ls_waldir();

-- 5. Detect severe table bloat
SELECT relname, n_dead_tup, n_live_tup,
       ROUND(n_dead_tup * 100.0 / NULLIF(n_live_tup + n_dead_tup, 0), 2) AS dead_tuple_ratio
FROM pg_stat_user_tables
WHERE n_dead_tup > 10000
ORDER BY dead_tuple_ratio DESC;
```

SAFETY PARAMETER

Configure `statement_timeout = '30s'` globally or per user to prevent runaway Cartesian queries from locking up database worker processes.

CHAPTER 20

Enterprise PostgreSQL Production Health Audit

PRE-FLIGHT ASSESSMENT

Before certifying any production PostgreSQL deployment, verify compliance against every standard in this enterprise DBA audit checklist.

- **Memory Baseline:** `shared_buffers` set to 25% RAM; `work_mem` sized conservatively against connection limits; Huge Pages configured.
- **Checkpoint Tuning:** `max_wal_size` ≥ 32GB; `checkpoint_completion_target = 0.9`; checkpoints triggered by timeout, not capacity.
- **Autovacuum Health:** Scale factor lowered to 0.05; cost limits increased to 2000; zero tables approaching 2-billion transaction freeze horizon.
- **Connection Pooling:** PgBouncer active in transaction mode; application microservices connect via pooler rather than direct backends.
- **Disaster Recovery:** pgBackRest physical backups executing daily; WAL archiving streaming continuously to cloud object storage; PITR tested quarterly.
- **High Availability:** Patroni cluster deployed across 3 availability zones with etcd consensus; split-brain failover validated under chaos testing.

AUTHOR CLOSING STATEMENT

PostgreSQL is one of the most reliable engineering feats in modern software. By configuring deliberate memory limits, tuning autovacuum aggressively, and backing your architecture with automated pooling and HA failover, your database will withstand millions of queries with rock-solid durability.

Datta Sable · Enterprise Analytics & Data Architect · *dattasable.com*